

CPU ↔ Bus ↔ Peripheral Interface Contract

Revision 2, 2026-09-26. Supersedes revision 1 (first draft reviewed with Aaron).

Two goals drive every change in this revision: match observable DMG behavior, and match what the CPU, interrupt controller, and timer now implement and verify (tb_timing 35/35, tb_interrupt_ctrl 32/32, tb_timer 41/41, Blargg 02-interrupts and instr_timing passing; full cpu_instrs regression in progress). Section 9 lists every change from revision 1 and why.

1. Core bus signals

One shared bus, one 100 MHz clock domain, two clock enables (§6). memory_map.v is the only module that decodes addresses and muxes read data. The CPU never talks to a peripheral directly.

Signal	Width	Driver	Description
addr	16	CPU	Address, held stable for the whole M-cycle
data_out	8	CPU	Write data, held stable for the whole M-cycle
data_in	8	bus	Read data, sampled by the CPU on the ce_m edge that ends the M-cycle
we	1	CPU	1 = write, held for the whole M-cycle; commits only on we && ce_m

There is no bus_stall. The DMG never inserts wait states: every CPU access takes exactly one M-cycle. Accesses the hardware blocks (VRAM in mode 3, OAM in modes 2 and 3, most of the bus during OAM DMA) complete on time but read \$FF or are dropped (§4).

The interrupt side channels (ie, if_reg, if_clear, if_clear_we) run directly between the CPU and interrupt_ctrl.v, not through memory_map.v (§5).

2. Address map (decode table in memory_map.v)

memory_map.v is pure routing and holds no storage. Each range is a separate module with the slave interface in §3.

Range	Owner	Notes
0000-00FF	boot ROM	Overlays the cartridge while FF50 has not been written
0000-7FFF	cartridge	ROM and MBC bank switching
8000-9FFF	PPU (VRAM)	Blocked in mode 3: reads \$FF, writes dropped (§4)
A000-BFFF	cartridge RAM	MBC-dependent, stretch
C000-DFFF	WRAM	
E000-FDFF	WRAM echo	Mirrors C000-DDFF through address decode, no second copy
FE00-FE9F	PPU (OAM)	Blocked in modes 2 and 3 and during OAM DMA (§4)
FEA0-FEFF	unusable	Behavior depends on PPU mode and revision (open item, §8)
FF00	joypad	
FF01-FF02	serial	Minimal unit: internal-clock transfers complete (§8)
FF03	unmapped	Reads \$FF
FF04-FF07	timer	DIV, TIMA, TMA, TAC; decode addr[15:2] == 14'h3FC1
FF08-FF0E	unmapped	Reads \$FF
FF0F	interrupt_ctrl	IF (§5)

Range	Owner	Notes
FF10–FF3F	APU	Stretch
FF40–FF45	PPU registers	LCDC, STAT, SCY, SCX, LY, LYC
FF46	OAM DMA unit	Separate from the PPU; bus master while a transfer runs (§4)
FF47–FF4B	PPU registers	BGP, OBP0, OBP1, WY, WX
FF50	boot ROM control	Any write unmaps the boot ROM; one-way until reset
FF51–FF7F	unmapped on DMG	Reads \$FF
FF80–FFFE	HRAM	Remains usable by the CPU during OAM DMA
FFFF	interrupt_ctrl	IE (§5)

Rules for every range:

- Unmapped addresses read \$FF, and writes to them are ignored.
- Unused bits inside a register read as 1 (for example IF bits 7–5, TAC bits 7–3, STAT bit 7). The owning module applies its own read mask; memory_map never does.

3. Peripheral slave interface

Every peripheral exposes the same interface so memory_map.v can mux them uniformly:

```
module <peripheral> (
    input wire      clk,
    input wire      rst,
    input wire      ce_gb,    // T-cycle tick (§6)
    input wire      ce_m,    // M-cycle tick, coincident with every 4th ce_gb
    input wire [15:0] addr,    // full bus address; peripheral decodes its own subrange
    input wire [7:0] data_in,  // write data: the CPU's data_out
    input wire      we,
    input wire      sel,      // 1 when addr falls in this peripheral's range
    output reg [7:0] data_out, // read data (see rules 2 and 3)
    output wire      stall    // reserved, always 0 on the DMG (§4)
);
```

Rules:

- **Writes commit exactly once**, on sel && we && ce_m. The CPU holds we for the whole M-cycle, so any other qualifier would commit a write up to four times.
- **Edge ordering: hardware first, then the CPU's access.** On every edge, a peripheral's own update (a tick, a mode change, a reload) happens before the CPU's access on that edge. A write on the same edge overrides the update unless the hardware documents otherwise (the TIMA reload is one such case). A read returns the value *after* that edge's update, so data_out is computed combinatorially from next-state values. This is the rule that made instr_timing pass.
- **Storage without autonomous updates** (ROM, WRAM, HRAM, VRAM/OAM contents) may use a registered or block-RAM read. Nothing changes those arrays on its own, so post-edge equals current, and the address is stable for the whole M-cycle, leaving far more than one clock of settling time. Any *decision* that depends on autonomous state, such as whether VRAM is blocked, must still use the post-edge value.
- data_out is 0 when sel is low.
- Naming: data_in here is the CPU's data_out. It is easy to cross these in gb_top.v.
- A module that needs only one enable may ignore the other; memory_map still passes both.

4. Blocked access and bus mastering (replaces the stall protocol)

The DMG CPU never waits for the bus. Revision 1 proposed stalling the CPU during VRAM/OAM-blocked modes; real hardware instead lets the access complete on time and ignores it. A stall would make a badly timed VRAM write succeed where a real DMG drops it.

Region	Blocked when	Blocked read	Blocked write
VRAM 8000-9FFF	PPU mode 3 (pixel transfer)	\$FF	dropped
OAM FE00-FE9F	PPU modes 2 and 3, and during OAM DMA	\$FF	dropped

- The blocked/not-blocked decision uses the PPU mode after the current edge (§3, rule 2).
- **OAM DMA (FF46).** A write starts a copy of 160 bytes from XX00-XX9F to OAM, one byte per M-cycle. While it runs, the DMA unit is the bus master for the external bus: HRAM remains usable by the CPU (which is why games run their DMA wait loop from HRAM), and the I/O registers stay reachable. The exact result of CPU accesses to other ranges during DMA, and the transfer's start delay, are open items (§8) to be settled with Mooneye's OAM DMA tests before implementation.
- **Known deviation, not modeled:** the DMG OAM corruption bug (accesses to FE00-FEFF, or 16-bit increments/decrements of a register pointing there, while the PPU is in mode 2). Stretch goal.
- The stall port stays in the slave interface as a reserved output tied to 0, so no module needs rewiring if a genuine need ever appears. The CPU has no stall input.

5. Interrupts

interrupt_ctrl.v (CPU side, owned by Jordie) is the sole owner of IE and IF. memory_map routes FF0F and FFFF to it with one sel; the module tells them apart by address.

- **IE (FFFF)** stores all 8 bits and reads them back as written.
- **IF (FF0F)** stores 5 bits. Bus reads return {3'b111, IF[4:0]}. The CPU-facing if_reg is {3'b000, IF[4:0]}, so ie & if_reg never sees phantom pending bits when software writes IE = \$FF.
- Writing a 1 to an IF bit is a real request (software interrupts); writing 0 clears it.

Request lines, one per source:

Signal	Driver	IF bit	Vector
irq_vblank	PPU	0 (highest priority)	\$0040
irq_lcdstat	PPU	1	\$0048
irq_timer	timer	2	\$0050
irq_serial	serial	3	\$0058
irq_joypad	joypad	4 (lowest priority)	\$0060

- **Strobe convention.** Each irq_* is high for exactly one 100 MHz clock: the clock whose ending edge is the event, and that edge must carry ce_gb. The peripheral derives the strobe combinationally from the same condition that drives its own state change, so the IF bit is set on the same edge as the event. A registered or held request would add a T-cycle of latency.
- **Same-clock priority:** bus write to IF, then CPU acknowledge, then peripheral requests. A request always survives a same-clock clear. This is a design choice, not documented hardware behavior; it guarantees an interrupt is never silently lost.
- **Acknowledge:** if_clear_we is held for exactly one M-cycle and commits on ce_m.

- **Dispatch** (in `cpu.v`) takes 5 M-cycles: two internal, push PC high, push PC low, load vector. The vector is selected at the start of the fourth M-cycle, after the high-byte push, and IF is acknowledged in that M-cycle. If the high-byte push overwrote IE and nothing enabled is still pending, the CPU jumps to \$0000 and clears nothing (the `ie_push` behavior).

6. Clock enables

Revision 1 gated every module with a single `ce` by wrapping its clocked block in `else if (ce)`. For the CPU that makes instruction length equal the number of FSM states, which is not how the SM83 is timed. It is replaced by two enables and a commit rule:

- **ce_gb**: one pulse per T-cycle, long-run average 4.194304 MHz. $100 \text{ MHz} / 4.194304 \text{ MHz} \approx 23.84$ is not an integer, so `clk_div` should use a fractional (phase) accumulator. Per-pulse jitter of one 100 MHz clock is invisible, because all DMG logic counts enables rather than measuring time. A fixed divide by 24 would run the whole system about 0.66% slow.
- **ce_m**: one pulse on every 4th `ce_gb`, on the same clock as that `ce_gb`. Both reset together, so the M-cycle phase is fixed from reset.
- The CPU's FSM runs on every 100 MHz clock. Only bus commits (fetch, read, write, stack, acknowledge) and internal M-cycles wait for `ce_m`.
- Peripherals run their T-cycle logic on `ce_gb` (timer counter, PPU dots, APU) and take CPU writes on `ce_m`.
- Alignment invariant: the timer's system counter has its low two bits at 00 on every `ce_m`. DIV writes commit on `ce_m` and reset the counter to 0, so the invariant survives them. `tb_timer` checks it continuously.
- The 25 MHz VGA enable is a separate pulse off the same 100 MHz clock. The DMG frame rate is about 59.73 Hz (70,224 T-cycles per frame), not 60 Hz, so the framebuffer decouples the two.

7. Migration status

Done (Jordie):

- `cpu.v`: `ce_m` input; every bus access commits on `ce_m`; all writes routed through one write state; internal M-cycles; late vector selection; no stall input.
- `interrupt_ctrl.v` and `timer.v` per \$3, \$5, \$6.
- Verification: `tb_timing` 35/35, `tb_interrupt_ctrl` 32/32, `tb_timer` 41/41, Blargg 02-interrupts and `instr_timing`.

To do (Aaron):

- `clk_div.v`: add `ce_m`; confirm or switch to a fractional ratio (\$6).
- `memory_map.v`: remove the IE/IF storage and IF-clear logic; become pure decode and mux per \$2; add selects for `interrupt_ctrl` (FF0F, FFFF), `timer` (FF04-FF07), `serial`, FF50, and the echo mirror; unmapped reads \$FF; pass writes through (peripherals commit on `ce_m`).
- `joypad.v`: change `irq_joypad` from a held pulse to the \$5 strobe; reads follow \$3 rule 2.
- `gb_top.v`: instantiate `interrupt_ctrl` and `timer`; wire the `irq_*` lines; distribute `ce_gb` and `ce_m`; remove any `bus_stall` wiring.
- PPU, when built: register file follows \$3 (post-edge reads, so LY and STAT read correctly on the edge they change); blocked access per \$4; VBlank and STAT requests as \$5 strobes.

8. Open items

Resolved from revision 1: the timer drives `irq_timer` as a §5 strobe; echo RAM is a mirror in address decode.

Still open:

- Confirm the PPU register file (FF40–FF45, FF47–FF4B) lives in `ppu/` behind the §3 interface, with FF46 in a separate DMA unit.
- OAM DMA: start delay, and the result of CPU accesses outside HRAM and I/O during a transfer. Settle with Mooneye's OAM DMA tests before implementing.
- FEA0–FEFF: read value per PPU mode on the DMG. Confirm from a primary source before implementing.
- Serial: a minimal unit where an internal-clock transfer completes after 8 bits at 8192 Hz, shifts in 1s (no cable), clears SC bit 7, and raises `irq_serial`. Without it, SC bit 7 never clears and any game waiting on a transfer hangs. Decide an owner.
- `clk_div` ratio: fixed or fractional (§6).
- Mooneye harness: the arbiter for behaviors this contract infers rather than quotes: the §3 edge ordering, `ie_push`, the timer reload interactions, and the exact T-cycle at which reads observe IF and TIMA changes.

9. Changes from revision 1

Revision 1	Revision 2	Reason
<code>bus_stall</code> ; VRAM/OAM stall the CPU	No stall; blocked reads \$FF, writes dropped	DMG never inserts wait states
Serial reads \$FF until implemented	Minimal serial unit	SC bit 7 would never clear; games waiting on a transfer hang
FF46 listed under PPU registers	Separate DMA unit, bus master	DMA is its own unit and takes over the bus
FF50 missing	Boot ROM control added	Part of the DMG map; <code>gb_top</code> already has a select for it
FEA0–FEFF reads \$FF	Open item	Behavior depends on PPU mode and revision
IE/IF stored in <code>memory_map</code>	<code>interrupt_ctrl</code> owns IE/IF	One owner resolves all IF writers; <code>memory_map</code> holds no state
<code>irq_*</code> single-cycle / one-ce pulse	One-clock strobe on the event edge	Single clock is missed; held pulse adds a T-cycle of latency
Set wins "matches real hardware"	Set wins, labeled a design choice	Not documented
One ce, else if (ce) wrap	<code>ce_gb</code> + <code>ce_m</code> , commit on <code>ce_m</code>	Wrap makes instruction length equal FSM state count
Registered peripheral reads	Post-edge (next-state) reads	Fixed the one-cycle error <code>instr_timing</code> detected
Fixed clock ratio implied	Fractional <code>ce_gb</code> recommended	100/4.194304 is not an integer